
SOAL 4: Rekayasa Ulang Prosedur Stat (25%)

a. Alasan Pilihan

Pilihan: Mengoreksi kekeliruan algoritma (Quick Fix)

Alasan berdasarkan konsep Reengineering:

1. Sesuai dengan Konsep Quick Fix Model (Basili, 1990)

Berdasarkan materi **Model Berorientasi Daur Ulang**, Quick Fix Model adalah pendekatan di mana:

- Source code dimodifikasi untuk menangani persoalan
- Perubahan penting dibuat cepat pada kode termasuk dokumentasi terkait
- Kode baru di-rekompilasi menghasilkan versi baru

Karakteristik kasus ini cocok untuk Quick Fix karena:

- Prosedur memiliki bug logika yang jelas dan terlokalisasi
- Tidak memerlukan investigasi awal yang kompleks seperti analisis dampak perubahan yang ekstensif
- Perbaikan dapat dilakukan langsung pada kode dengan cepat

2. Jenis Perubahan yang Sesuai: Recode

Menurut klasifikasi **Jenis Perubahan dalam Reengineering:**

- **Recode:** Sifat implementasi program diubah dengan kode ulang, bisa rephrasing atau translasi
- **Rephrasing:** normalisasi, optimisasi, refactoring, renovasi

Perbaikan algoritma prosedur Stat termasuk kategori **Recode - Rephrasing** karena:

- Mengubah sifat implementasi (inisialisasi dan logika loop)
- Melakukan normalisasi kode (memperbaiki logika yang salah)
- Tidak mengubah level abstraksi sistem

3. Strategi Reengineering: Rework

Berdasarkan **Strategi Rekayasa Ulang - Rework:**

- Menerapkan 3 prinsip: abstraction, alteration, refinement
- Maksud proyek menstrukturkan konstruksi aliran kendali

- Ancangan klasik rework:
 1. **Aplikasi abstraksi:** memahami struktur kode (parsing, membuat CFG)
 2. **Aplikasi alterasi:** menerapkan perbaikan logika
 3. **Aplikasi refinement:** menghasilkan kode yang diperbaiki

Kasus prosedur Stat menggunakan Rework karena:

- Memerlukan abstraksi untuk memahami alur logika (inisialisasi → iterasi → perhitungan)
- Melakukan alterasi pada logika inisialisasi dan kondisi
- Refinement menghasilkan kode yang lebih benar secara logika

4. Mengapa TIDAK Memecah Prosedur (Redesign)?

Redesign melibatkan perubahan sifat desain software seperti:

- Restrukturisasi arsitektur
- Modifikasi model data sistem
- Mengganti prosedur/algoritma lebih efisien

Alasan tidak memilih pemecahan prosedur:

a. Single Responsibility Principle Terpenuhi

- Prosedur memiliki satu tanggung jawab koheren: menghitung statistik dasar (minimum dan rata-rata)
- Kedua operasi (mencari min dan menghitung avg) saling terkait dalam konteks statistik

b. Efisiensi Komputasi

- Menggabungkan operasi dalam satu loop: $O(n)$
- Memisahkan menjadi dua prosedur: $O(2n)$
- Untuk quick fix, efisiensi lebih diutamakan

c. Tidak Ada Perubahan Fundamental

- Bug hanya pada implementasi, bukan pada desain arsitektur
- Tidak ada kebutuhan untuk mengubah interface atau pemanggilan prosedur

d. Prinsip Alterasi (Alteration Principle)

- Menurut konsep reengineering, alterasi adalah "opsi non esensial pembuatan perubahan representasi sistem"
- Alterasi tidak mengubah derajat abstraksi
- Perbaikan bug termasuk alterasi, bukan redesign

5. Bug yang Ditemukan (Hasil Code Reverse Engineering):

Berdasarkan Teknik Reverse Engineering - Control Flow Analysis:

- **Bug logika inisialisasi:**
 - $\text{indeks} \leftarrow 0$ menyebabkan iterasi dimulai dengan $\text{indeks} \leftarrow 1$ (benar)
 - Namun nilai $\text{Min} \leftarrow 999999999$ adalah arbitrary value, tidak robust
- **Kelemahan lain:**
 - Tidak ada penanganan untuk array kosong
 - Nilai inisialisasi Min tidak menggunakan elemen array yang valid
 - Pembagian $\text{Avg} \leftarrow \text{Sum} / \text{indeks}$ akan menggunakan nilai 100 (sudah benar, tetapi kurang eksplisit)

b. Revisi Prosedur (Hasil Reengineering)

Berdasarkan **Proses Rekayasa Ulang - Fase Renovasi**, sistem kini dimodifikasi dengan fokus pada representasi sistem (source code). Berikut revisi menggunakan pendekatan **Rework Strategy**:

```
PROCEDURE Stat(input A:array[1...100] of integer, output Min:integer,
               output Avg:real)
{ I.S : tabel A terisi 100 buah elemen integer positif }
{ F.S : nilai Min dan Avg terdefinisi }
{ Proses : menentukan nilai terkecil elemen tabel (Min) dan rata-rata (Avg) }
```

KAMUS LOKAL

```
    indeks : integer    {posisi elemen tabel}
    Sum : integer        {jumlah nilai elemen tabel A}
    N : integer          {banyaknya elemen, konstanta = 100}
```

ALGORITMA

```
{ Konstanta }
N ← 100

{ Inisialisasi dengan elemen pertama array }
Min ← A[1]
Sum ← A[1]
indeks ← 2

{ Iterasi untuk elemen ke-2 sampai ke-N }
while (indeks ≤ N) do
    Sum ← Sum + A[indeks]
    if (A[indeks] < Min) then
        Min ← A[indeks]
    endif
    indeks ← indeks + 1
endwhile

{ Hitung rata-rata }
Avg ← Sum / N
```

Penjelasan Perbaikan (Dokumentasi Perubahan):

Sesuai dengan **Fase Redokumentasi** dalam Phase Reengineering Model:

1. Perbaikan Inisialisasi (Normalization)

- **Sebelum:** $Min \leftarrow 999999999$ (nilai arbitrary yang tidak aman)
- **Sesudah:** $Min \leftarrow A[1]$ (menggunakan elemen pertama array sebagai nilai awal)
- **Alasan:** Nilai 999999999 tidak robust; jika ada elemen > 999999999 , algoritma gagal

2. Perbaikan Logika Sum (Correction)

- **Sebelum:** $Sum \leftarrow 0$, mulai akumulasi dari $A[1]$
- **Sesudah:** $Sum \leftarrow A[1]$, mulai iterasi dari $A[2]$
- **Alasan:** Menghindari akses berlebihan dan membuat logika lebih konsisten

3. Perbaikan Kondisi Loop (Restructuring)

- **Sebelum:** `while (indeks < 100) do` dengan $indeks \leftarrow 0$ awal
- **Sesudah:** `while (indeks <= N) do` dengan $indeks \leftarrow 2$ awal
- **Alasan:** Lebih eksplisit, menggunakan konstanta N untuk maintainability

4. Perbaikan Perhitungan Rata-rata (Clarity)

- **Sebelum:** $Avg \leftarrow Sum / indeks$ (menggunakan variabel loop)
- **Sesudah:** $Avg \leftarrow Sum / N$ (menggunakan konstanta eksplisit)
- **Alasan:** Lebih jelas dan menghindari ketergantungan pada nilai akhir variabel loop

5. Perbaikan Perbandingan (Readability)

- **Sebelum:** `if (Min > A[indeks]) then`
- **Sesudah:** `if (A[indeks] < Min) then`
- **Alasan:** Lebih intuitif membaca "jika elemen lebih kecil dari Min"

6. Penambahan Konstanta N (Maintainability)

- Menambahkan konstanta $N \leftarrow 100$ untuk meningkatkan keterawatan
- Jika ukuran array berubah, hanya perlu mengubah satu tempat

Klasifikasi Reengineering yang Dilakukan:

Berdasarkan **Source Code Reengineering Reference Model (SCORE/RM)**:

1. **Encapsulation:** Membuat baseline kode awal (prosedur Stat versi lama)
 2. **Transformation:** Mengubah alir kendali agar kode lebih terstruktur (kondisi loop, inisialisasi)
 3. **Normalization:** Menyelidiki data dan strukturnya (variabel Min, Sum, indeks)
 4. **Interpretation:** Memahami maksud perangkat lunak (menghitung Min dan Avg)
 5. **Regeneration:** Implementasi ulang kode sumber dengan logika yang benar
 6. **Certification:** Evaluasi kode sumber baru (harus diuji untuk memastikan hasil sama untuk input valid)
-

Testing dan Verification (Fase Uji Sistem Target):

Untuk memastikan perbaikan benar, perlu dilakukan pengujian dengan berbagai kasus:

Test Case 1: Array dengan nilai yang meningkat

- Input: $A = [1, 2, 3, \dots, 100]$
- Expected: Min = 1, Avg = 50.5

Test Case 2: Array dengan nilai yang menurun

- Input: $A = [100, 99, 98, \dots, 1]$
- Expected: Min = 1, Avg = 50.5

Test Case 3: Array dengan nilai sama

- Input: $A = [50, 50, 50, \dots, 50]$ (100 elemen)
- Expected: Min = 50, Avg = 50

Test Case 4: Array dengan nilai ekstrem

- Input: A dengan elemen terkecil = 1, terbesar = 1000000000
 - Expected: Min = 1 (bukan terbatas 999999999)
-

Kesimpulan: Pilihan mengoreksi kekeliruan algoritma menggunakan **Quick Fix Model dengan strategi Rework** adalah pendekatan yang tepat karena masalahnya adalah bug implementasi yang terlokalisasi, bukan masalah desain arsitektural. Pendekatan ini sejalan dengan prinsip reengineering yang efisien untuk perbaikan cepat tanpa mengubah struktur fundamental sistem.

Tentu, berikut adalah jawaban untuk soal nomor 4, dengan memanfaatkan konsep dari materi "Rekayasa Ulang" (EPL06-Rekayasa Ulang.pdf) yang Anda berikan.

a. Uraikan Alasan Pilihan Anda!

Pilihan yang diambil adalah **memecah prosedur menjadi dua**.

Alasan:

Pilihan ini didasarkan pada tujuan fundamental dari rekayasa ulang (re-engineering), yaitu untuk **meningkatkan mutu** ¹¹ dan **keterawatan (maintainability)** ² perangkat lunak, bukan sekadar memperbaiki *bug*.

1. **Masalah Desain (Low Cohesion):** Prosedur Stat yang ada saat ini melanggar prinsip desain yang baik karena memiliki **dua tanggung jawab** sekaligus: (1) mencari nilai terkecil (Min) dan (2) menghitung nilai rata-rata (Avg). Ini disebut memiliki **kohesi fungsional yang rendah**.
2. **Tujuan Rekayasa Ulang:** Materi "Rekayasa Ulang" menyatakan bahwa salah satu ancangan (evolusioner) bertujuan membangun sistem baru dengan **komponen yang kohesif secara fungsional** ³, yang akan menghasilkan **desain baru yang lebih kohesif** ⁴.
3. **Jenis Perubahan:**
 - o Mengoreksi kekeliruan algoritma (pembagian integer) hanya termasuk Recode (kode ulang), seperti *refactoring* ⁵. Ini adalah *quick fix* yang hanya memperbaiki gejala, bukan masalah desainnya.
 - o Memecah prosedur menjadi dua adalah tindakan Redesign (desain ulang) ⁶. Tindakan ini secara fundamental mengubah desain prosedur untuk meningkatkan kualitasnya.
4. **Manfaat:** Dengan memecah prosedur, kita mendapatkan dua prosedur baru yang sangat kohesif (masing-masing hanya punya satu tugas). Ini membuat prosedur-prosedur baru tersebut lebih mudah dipahami, dirawat ⁷, dan **digunakan ulang (reusable)** ⁸ di bagian lain program secara terpisah.

Meskipun "mengoreksi algoritma" lebih cepat, "memecah prosedur" adalah pilihan rekayasa ulang yang lebih tepat karena secara aktif meningkatkan desain dan mutu struktural perangkat lunak.

b. Buatlah Revisi Prosedur Tersebut!

Berikut adalah revisi prosedur yang telah dipecah menjadi dua, di mana masing-masing memiliki satu tanggung jawab yang jelas.

Prosedur 1: Mencari Nilai Minimum

Cuplikan kode

```
PROCEDURE FindMin(input A:array[1...100] of integer, output Min:integer)
{ I.S : tabel A terisi 100 buah elemen integer positif }
{ F.S : nilai Min terdefinisi sebagai nilai terkecil di dalam tabel A }
```

KAMUS LOKAL

indeks : integer {posisi elemen tabel}

ALGORITMA

indeks $\Downarrow$ 0

Min $\Downarrow$ 999999999 {Inisialisasi dengan nilai yang sangat besar}

```
while (indeks < 100) do
  indeks  $\Downarrow$  indeks + 1
  if (Min > A[indeks]) then
    Min  $\Downarrow$  A[indeks]
  endif
endwhile
{endwhile: indeks=100}
```

Prosedur 2: Menghitung Nilai Rata-rata

(Prosedur ini juga sekaligus **mengoreksi kekeliruan algoritma** pada kode aslinya, yaitu *integer division*.)

Cuplikan kode

```
PROCEDURE CalculateAvg(input A:array[1...100] of integer, output Avg:real)
{ I.S : tabel A terisi 100 buah elemen integer positif }
{ F.S : nilai Avg terdefinisi sebagai nilai rata-rata elemen tabel A }
```

KAMUS LOKAL

indeks : integer {posisi elemen tabel}

Sum : integer {jumlah nilai elemen tabel A}

ALGORITMA

indeks $\Downarrow$ 0

Sum $\Downarrow$ 0

```
while (indeks < 100) do
  indeks  $\Downarrow$  indeks + 1
  Sum  $\Downarrow$  Sum + A[indeks]
endwhile
{endwhile: indeks=100}
```

{ KOREKSI ALGORITMA:

Pembagian 'Sum' (integer) dengan 'indeks' (integer) akan menghasilkan integer. Kita perlu 'cast' salah satunya ke 'real' agar hasilnya 'real'. }

Avg $\Downarrow$ cast(Sum, real) / indeks